FR

Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.

Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

> *RLS (Row-Level Security) est la dernière ligne de défense entre un attaquant et les données utilisateurs. Sans elle, n'importe qui ayant la clé publique Supabase pourrait* `SELECT * FROM profiles` *et exfiltrer toute la base. Avec elle, même un endpoint API mal écrit ne peut pas violer la sécurité.*

PHILOSOPHIE

Principe : default-deny

RÈGLE ABSOLUE

Sur Sakata, **toute table contenant des données utilisateur a RLS activée**. Les policies ajoutent ensuite des permissions explicites. Aucune table user-touchée ne reste « ouverte ».

CODE SQL

```sql
-- Sur chaque nouvelle table :
ALTER TABLE public.ma_table ENABLE ROW LEVEL SECURITY;

-- Sans aucune policy ensuite : NOTHING is accessible
-- (sauf via service_role key, contournement explicite)
```

RÉUTILISABLES

Patterns standards

1. Owner-only (le plus courant)

La table contient des données qui n'appartiennent qu'à un seul user. Exemples : `admin_notes`, `ecole_scores`, `push_subscriptions`.

CODE SQL

```sql
-- ALL = SELECT + INSERT + UPDATE + DELETE en une policy
CREATE POLICY "owner_only"
  ON ma_table FOR ALL
  TO authenticated
  USING (user_id = auth.uid())
  WITH CHECK (user_id = auth.uid());
```

2. Public-read, owner-write

Lecture publique (ou pour authentifiés), écriture restreinte au propriétaire. Exemples : `profiles`, `articles` (sauf premium).

CODE SQL

```sql
CREATE POLICY "public_read"
  ON profiles FOR SELECT
  TO public USING (true);

CREATE POLICY "self_update"
  ON profiles FOR UPDATE
  TO authenticated
  USING (id = auth.uid())
  WITH CHECK (id = auth.uid());

-- INSERT par trigger après auth.signup, pas par user direct
-- DELETE jamais autorisé pour le user (admin uniquement)
```

3. Role-based

Permission dépendante du rôle dans `profiles.role`. Exemples : tables admin, modération.

CODE SQL

```sql
CREATE POLICY "admin_or_manager"
  ON contribution_requests FOR UPDATE
  TO authenticated
  USING (
    EXISTS (
      SELECT 1 FROM profiles
      WHERE profiles.id = auth.uid()
        AND profiles.role IN ('admin', 'manager')
    )
  );
```

4. Time-windowed (édition limitée dans le temps)

Le user peut modifier son message pendant 5 minutes seulement.

CODE SQL

```sql
CREATE POLICY "edit_within_5_min"
  ON chat_messages FOR UPDATE
  TO authenticated
  USING (
    sender_id = auth.uid()
    AND created_at > NOW() - INTERVAL '5 minutes'
  );
```

5. Premium gating

CODE SQL

```sql
CREATE POLICY "premium_articles"
  ON articles FOR SELECT
  TO authenticated
  USING (
    requires_premium = false
    OR EXISTS (
      SELECT 1 FROM profiles
      WHERE profiles.id = auth.uid()
        AND profiles.subscription_tier = 'premium'
    )
  );
```

DRY

Helpers réutilisables

MIGRATIONS/SECURITY_HELPERS.SQL SQL

```sql
-- Vérifier le rôle de l'utilisateur courant
CREATE OR REPLACE FUNCTION public.is_admin_or_manager()
RETURNS BOOLEAN
LANGUAGE sql
STABLE
SECURITY DEFINER
AS $$
  SELECT EXISTS (
    SELECT 1 FROM profiles
    WHERE id = auth.uid()
      AND role IN ('admin', 'manager')
  );
$$;

-- Usage simplifié dans les policies :
CREATE POLICY "managers_can_lock"
  ON forum_threads FOR UPDATE
  TO authenticated
  USING (is_admin_or_manager());
```

BONUS PERFORMANCE

Les fonctions `STABLE` sont memoizées par Postgres au sein d'une même requête → un seul lookup même si la policy est évaluée 1000 fois.

ERREURS COURANTES

Anti-patterns à proscrire

✕ NE JAMAIS FAIRE

- RLS désactivée « pour aller plus vite » sur une table user-touchée.
- Faire confiance au service_role key dans une API route sans vérifier l'auth de l'utilisateur en amont.
- Policy avec `USING (true)` sur une table sensible.
- Oublier `WITH CHECK` sur INSERT/UPDATE — la policy lit mais ne valide pas l'écriture.
- Comparer `auth.role()` au lieu de `auth.uid()` + lookup profile.

VALIDATION

Tests de non-régression

Comment savoir qu'une nouvelle migration n'a pas cassé une RLS ? Tests scriptés avec deux clients Supabase :

CODE TYPESCRIPT

```typescript
// tests/rls/profiles.test.ts
const userA = createBrowserClient(URL, ANON_KEY);
const userB = createBrowserClient(URL, ANON_KEY);

test("user A cannot UPDATE user B's profile", async () => {
  await userA.auth.signInWithPassword({ email: "a@x.com", ... });
  const { error } = await userA
    .from("profiles")
    .update({ username: "hacked" })
    .eq("id", USER_B_ID);

  expect(error).toBeTruthy();  // Doit échouer
});
```

DISCIPLINE

Checklist nouvelle table

1. Activer RLS : `ALTER TABLE ... ENABLE ROW LEVEL SECURITY`.
2. Identifier le pattern (owner-only, role-based, etc.).
3. Écrire les policies SELECT, INSERT, UPDATE, DELETE séparément (granularité).
4. Penser au `WITH CHECK` sur INSERT/UPDATE.
5. Ajouter index sur `user_id` et autres colonnes filtrées.
6. Écrire un test de non-régression.
7. Documenter dans la migration le pourquoi des policies.

Sakata.com — Documentation interne · architecture-rls-supabase · v2026.4

« Moto oyo ayebi te kotombola miso na likolo, akoki te koyeba esika oyo mbula eutaka. » — Proverbe Sakata

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM